Master 2 Informatique - IA2VR

TMI - TRAITEMENT ET ANALYSE D'IMAGES

Partie 1 : Analyse des contours et des formes dans les images

Partie II à IV à réaliser et à déposer (**code plus compte-rendu)** sur arche ou à envoyer par mail à *debled@loria.fr* – Penser à bien documenter votre code –
Tous les textes en bleu sont des questions dont les réponses sont à écrire dans le compte-rendu.

ATTRIBUTS GEOMETRIQUES DE CONTOURS

Le but de cette étude est d'analyser un contour discret 2D : calcul des tangentes, normales, courbure en chaque point, calcul de la longueur du contour, extraction des points dominants.

I. IMAGES ET CODE FOURNIS

Sont fournis :

- des images de test : *formeTest.pgm, disque.pgm, blob.pgm, etoile1.pgm, etoile2.pgm*
- des classes python ou c++ de base pour l'analyse d'un contour discret dans les fichiers .cpp et .h : FreemanCode, Path (à compléter), DigitalLineSegment, DigitalTangent et DigitalTangentPositive,
  En c++ :
- un fichier exécutable permettant de visualiser un contour discret : Contour_Analysis.cpp,
- un fichier makefile permettant de compiler l'ensemble des fichiers.
  En python :
- un fichier main.py permettant de visualiser un contour discret

Ci-dessous vous trouverez le détail des classes c++, les classes fournies en python ont les mêmes fonctionnalités et les mêmes noms.

Classe FreemanCode

Cette classe correspond au code de Freeman d'un chemin discret 4-connexe. Elle contient donc le premier point du chemin et la suite des déplacements élémentaires permettant de le parcourir.
Ses méthodes sont les suivantes :

1. Le constructeur **`FreemanCode(Mat ip, int gl)`** traite une image en niveaux de gris et construit le code de Freeman du contour interpixel de la première composante 8-connexe de niveau de gris gl rencontrée lors d'un parcours gauche-droite/haut-bas de l'image.
2. **`int getLength()`** : longueur du chemin (nombre de déplacements élémentaires du code).
3. **`Point getP0()`** : premier point du chemin.
4. **`int getDeltaX (int ind),int getDeltaX (int ind)`** : variations en x et y pour un pas du chemin discret. L'indice ind doit vérifier 0 <= ind < longueur.
5. **`int getCode (int ind, int shift)`** : direction élémentaire sur un chemin supposé fermé.

Classe Path

Cette classe contient la suite des points discrets d'un chemin 4-connexe (en général le contour interpixel d'une région 8-connexe du plan). Elle contient la classe interne ContourPoint :

```
// dans le cas d'un contour interpixel (_x,_y) désigne le point demi-entier (_x+1/2, _y+1/2)
// l'axe des x étant orienté vers la droite et celui des y vers le haut
class ContourPoint{
	int _x;
	int _y;
	// vecteur tangent correspondant à la tangente symétrique au point (_x,_y)
	int _tx;
	int _ty;
	// vecteur tangent  dans la direction positive
	int _tpx;
	int _tpy;
	int _lgrTp; // Nombre de points de la tangente positive
}
```

La classe **Path** contient les méthodes suivantes :

1. Le constructeur **`Path(FreemanCode)`** qui construit la suite des points d'un chemin discret à partir de son code de Freeman et qui calcule le vecteur directeur de la **tangente symétrique** en chacun de ces points ainsi que le vecteur directeur de **la tangente positive**.
2. **`int getLength()`** : longueur du chemin en nombre de points.
3. **`int getContourX(int ind), int getContourY(int ind)`** renvoient l'abscisse et l'ordonnée du point d'indice ind du contour.
4. **`int getContourTX(int ind), int getContourTY(int ind)`** renvoient les deux composantes du vecteur tangent symétrique calculé au point d'indice ind du contour.
5. **`int getContourTPX(int ind), int getContourTPY(int ind)`** renvoient les deux composantes du vecteur de la tangente positive calculée au point d'indice ind du contour.
6. **`int getContourTPLgr(int ind),`** renvoie le nombre de points de la tangente positive calculée au point d'indice ind du contour.
7. **`int getPMin(), int getPMax()`** renvoient les deux coins opposés de la boîte englobante du contour.

Classes utilisées pour le calcul des tangentes :

- La classe **DigitalLineSegment** représente un segment de droite discrète 4-connexe du premier quadrant, codé suivant sa définition arithmétique. Elle contient plusieurs constructeurs et la méthode **`boolean addPositivePoint(int x, int y)`** permettant d'ajouter un point à l'extrémité droite d'un segment 4 connexe du premier quadrant selon le théorème vu en cours.
- La classe **DigitalTangent** correspond à une partie linéaire d'un contour discret, centrée autour d'un point de contour (tangente symétrique).
- La classe **DigitalTangentPositive** correspond à une partie linéaire d'un contour discret, débutant sur un point de contour (tangente dite positive en ce point) en ajoutant les points dans le sens du parcours.

Compiler l'ensemble de ces classes et visualiser les résultats obtenus avec le programme Contour_Analysis (ou main.py en python) sur les différentes images.

Ce programme permet de visualiser le contour 4-connexe formé de pointels d'un objet discret présent dans une image. De plus, la normale issue du calcul de la tangente dite positive (c'est-à-dire la plus longue droite discrète dans le sens de parcours à partir du point courant) est représentée tous les cinq points.
Ce programme affiche dans la console, pour chaque point du contour, son indice et l'indice du dernier point de sa tangente positive ainsi que les caractéristiques (b,a) du segment de droite associé.

II. CALCUL DES TANGENTES SELON L'ESTIMATEUR λ-MST

L'objectif de cette partie est de calculer le vecteur tangent en chaque point d'un contour 4-connexe selon l'estimateur λ-MST. Pour cela plusieurs étapes sont proposées.

1. **Calcul de la couverture tangentielle** qui est la suite de tous les segments maximaux présents sur le contour. Vous afficherez pour chaque segment maximal l'indice dans le contour du premier et du dernier point du segment ainsi que les caractéristiques du segment (a,b).
   Remarque : un calcul optimal de la couverture tangentielle n'est pas demandé, vous pouvez utiliser en chaque point du contour les indices de début et de fin des tangentes positives et en déduire les indices de début et fin des segments maximaux.

   Indiquer l'algorithme utilisé en pseudo-code.
   Afficher et commenter les résultats obtenus pour l'image *testforme.pgm*.

2. **Calcul des tangentes selon l'estimateur λ-MST**. Déduire de la couverture tangentielle un algorithme pour calculer le vecteur tangent en chaque point en considérant tous les segments maximaux passant par le point considéré (voir formule dans le cours, section III.2). Le programmer et visualiser les résultats obtenus (normales en chaque point) dans le programme **Contour_Analysis (en c++ ou main.py en python)**.

   Pour l'image *testforme.pgm*, vous indiquerez dans le compte-rendu les détails des calculs réalisés et les vecteurs tangents obtenus aux points d'indice 20, 48 et 49.

   Les images obtenues pour les images tests devront apparaître dans le compte-rendu.

   Plus spécifiquement, pour l'image *etoile2.pgm*, choisir 5 points pertinents que vous indiquerez sur l'image résultat en changeant la couleur de la normale en ces points, et pour ces points remplir le tableau ci-dessous :

Coordonnées du point ou indice	Pente de la normale obtenue avec la tangente positive	Pente de la normale obtenue avec l'estimateur λ-MST

…	…	…

3. Expliquez le **protocole expérimental** utilisé dans les deux publications [LVV07, VL09] mises à disposition sur arche pour comparer les différents estimateurs de tangente.

   Quelles sont les conclusions des auteurs sur les différentes méthodes. Obtenez-vous des résultats similaires ?

   [LVV07] J.-O. Lachaud, A. Vialard, F. de Vieilleville, *Fast, accurate and convergent tangent estimation on digital contours*, in Image and Vision Computing, vol 25, 2007.
   [VL09] F. de Vieilleville, J.-O. Lachaud, *Fast, accurate and convergent tangent estimation on digital contours Comparison and improvement of tangent estimators on digital curves*, in Pattern Recognition, vol 42, 2009.

III. CALCUL DE LA LONGUEUR DES CONTOURS

1. En utilisant les extrémités des tangentes positives, programmer un algorithme de polygonalisation d'un contour.

   Indiquer l'algorithme utilisé en pseudo-code.
   Afficher le polygone obtenu sur les images test.

2. En déduire un estimateur de la longueur d'un contour.

   Indiquer les longueurs obtenues pour chacun des contours des images tests

IV. CALCUL DE LA COURBURE

Lire la partie III.3. du cours et choisir un des estimateurs de courbure proposé.

1. Enrichir la classe **ContourPoint** avec une information locale supplémentaire : la courbure calculée en ce point selon la méthode que vous aurez choisie.

2. Visualiser le profil de courbure obtenu.
   Pour cela, vous pouvez par exemple utiliser gnuplot (http://www.gnuplot.info/). Supposons que le fichier data contienne la suite des courbures d'un contour de la façon suivante :
   0 0.209415
   1 0.210887
   2 0.197836
   3 0.174762
   4 0.147319
   ...

Le profil de courbure peut alors être affiché avec :
echo "plot 'data' with lines" | gnuplot -persist

Commenter les résultats obtenus sur les images fournies. Quelles formes pourraient être ajoutées pour faire une série de tests pertinents ?

V. **POINTS DOMINANTS** (question optionnelle)

Utilisez le profil de courbure pour sélectionner puis visualiser les points caractéristiques d'un contour : il s'agit des extrema locaux du profil de courbure. On peut ne sélectionner ces extrema locaux que parmi les points de courbure supérieure en valeur absolue à un seuil prédéfini.
Commenter les résultats obtenus et vos choix de seuils sur les différentes images de test.